

AI-Powered Document & Knowledge Assistant (RAG)

Project Documentation — what it is, why it matters, and how to build it

1. What Is It?

A web application that lets a user upload their own documents (PDFs, Word files, text files) and then ask natural-language questions about that content — receiving direct, grounded answers instead of a list of search results to sift through.

It is commonly known by a few names:

- “Chat with your documents” tool
- AI knowledge assistant
- RAG system — Retrieval-Augmented Generation

At a high level, it is an AI layer placed on top of a private document collection, so the AI can accurately answer using only the content that was actually uploaded — rather than guessing from general internet knowledge it was trained on.

2. Why We Need It

- General-purpose AI models (ChatGPT, Claude, etc.) are trained on public data up to a fixed cutoff date. They have never seen a company's private, internal, or newly created documents.
- Manually digging through long documents to find one answer is slow, and most users give up rather than read 40 pages to find a single paragraph.
- Support and documentation teams answer the same repeated questions daily, even when the answer already exists somewhere in the docs.
- Businesses increasingly want fast, accurate self-service — but pointing users at raw ChatGPT risks confidently wrong answers, which is a trust and liability problem, not just a UX one.

Example: a SaaS company's customers repeatedly email support asking questions already answered somewhere in a 200-page documentation site. A RAG assistant lets customers get an accurate answer instantly, sourced from the real docs — cutting repetitive ticket volume without sacrificing accuracy.

3. What Problem It Solves

Problem	How RAG Solves It
AI hallucination (confidently wrong answers)	Answers are grounded in retrieved source text, not model memory

Problem	How RAG Solves It
AI has no knowledge of private or internal data	Documents are indexed and searched at query time, so the AI effectively “reads” the right content before answering
Limited context window	Only the relevant chunks are sent to the model, not the entire document set
Stale or scattered knowledge	Re-indexing on document update keeps answers current; no separate FAQ page to maintain by hand
Findability across large document sets	Semantic retrieval surfaces the right content even when the user's wording doesn't match the document's wording

4. Why Not Just Store Documents and Use a Search Option?

This is the question every stakeholder should ask before approving the project — and it is a fair one. Plain search is not obsolete; it is just the wrong tool for certain situations.

Where plain search is genuinely fine:

- Small document sets (a handful of files) a person can reasonably skim
- Exact-match lookups — a filename, an error code, a function name
- Situations where cost and simplicity matter more than convenience

Where keyword search breaks down, and RAG doesn't:

- **Wording mismatch.** Keyword search matches literal words. If a policy document says “termination clause” and a user searches “how do I cancel,” keyword search returns nothing. RAG matches by meaning, not exact text, so it finds the right passage regardless of phrasing.
- **Search returns documents; RAG returns answers.** A search engine hands back a ranked list of files that might contain the answer — the user still has to open each one and read it. RAG does that reading and synthesis step for them.
- **Multi-source synthesis.** If an answer requires combining information from three different sections, keyword search returns three separate hits and leaves the user to piece them together. RAG can pull from multiple sources and generate one coherent answer.
- **No memory of the conversation.** Every search is an isolated lookup. A RAG assistant can handle natural follow-up questions because it retains context — plain search cannot.
- **Scale.** With 10 documents, manual search-and-read works fine. With 10,000 documents, it doesn't — but retrieval speed in a properly indexed RAG system barely changes with scale.

Honest caveat: RAG is not a strict upgrade in every case. It costs more per query (embedding + LLM calls), adds infrastructure (vector database, ingestion pipeline), and is harder to debug than a plain search index when something goes wrong. Use plain search when queries are simple and exact; use RAG when users ask natural-language questions across a large or loosely organized document set.

5. Tech Stack

Layer	Technology	Purpose
Frontend + API	Next.js (App Router) + TypeScript	UI and API routes for upload, chat, and auth
Authentication	OAuth 2.0 (Google)	Keeps each user's documents private to their account
File storage	AWS S3	Stores original uploaded files
Document parsing	pdf-parse, or Python (PyPDF2 / unstructured)	Extracts raw text from uploaded files
Chunking	Custom splitter or LangChain text splitter	Breaks text into overlapping, meaningful chunks
Embeddings	OpenAI Embeddings API	Converts text chunks and queries into vectors
Vector storage	MongoDB Atlas Vector Search	Stores and searches vectors by semantic similarity
Answer generation	OpenAI API (GPT-4o-mini class model)	Generates the grounded answer from retrieved chunks
Real-time delivery	WebSockets	Streams the answer token-by-token instead of a blocking wait
Notifications	Resend	Emails the user when a large document finishes processing
Background jobs (optional)	AWS SNS / SQS	Queues document ingestion so upload doesn't block on parsing large files
Security	CloudFront + signed URLs, rate limiting	Protects file access and public endpoints

6. MVP — What to Build First

Building the entire system above in one pass is a mistake — a working small version beats an ambitious broken one, and it's the version you can actually finish, deploy, and defend in an interview. Split the build into two phases.

Phase 1 — MVP (solo build, roughly 1–2 weeks)

- Single-format upload (PDF only)
- Store the original file in S3
- Parse text synchronously — no job queue yet
- Fixed-size chunking with overlap

- Generate embeddings via OpenAI and store them in MongoDB Atlas Vector Search
- Basic chat UI: question → embed → similarity search → top-k chunks → GPT-generated answer
- Show which source chunk or document the answer came from — this alone is a strong signal, since it proves the answer is grounded rather than generated blind

Phase 2 — After the MVP Works

This phase is what demonstrates growth and system-thinking in an interview — it's the difference between “I followed a tutorial” and “I understood the production problem.”

- OAuth and multi-user isolation
- Multi-format support (docx, txt, web pages)
- Async ingestion via SQS so large files don't block the upload response
- WebSocket streaming for the chat response
- Rate limiting and CloudFront / Turnstile protection on public endpoints
- A re-ranking step to improve retrieval accuracy
- A small evaluation set to measure whether retrieval is actually returning the right chunks